ANSWER KEY · AGENTIC HUNTING WORKSHOP

# Module 3 — Watch the Research Agent (35 min, instructor demo)

> **You watch; the instructor drives.** Claude (an AI assistant) runs on the presenter's laptop. It reads a research doc and your hunts, opens a new hunt from the research, then **writes** the Splunk searches one at a time. In this module Claude is **not connected to Splunk**: the presenter reads each search, pastes it into Splunk Web, and pastes the result rows back to Claude. (Next module, Claude runs them itself.) You don't need an LLM key. If you want, paste the same searches into your own Splunk (time picker: **Last 30 days**).
> **Level coverage: L3.** Starting point: **H-0005**, the beacon you left open in Module 1. Ends with **H-0007** on the presenter's screen.

> **Answer key — staff + self-study**
>
> **Presenter prep (before class, ~10 min).**
>
> 1. Demo workspace, outside `~/Documents/Nebulock` (Claude Code also reads parent `CLAUDE.md` files): `staff/demo-workspace.sh <dir> "<your name>"`. By hand it is: `athf init --non-interactive`; copy `HUNTS_EXPECTED/H-0004.md`, `H-0005.md` (Module 1) and `H-0006.md` (Module 2) into `hunts/`; then the research doc: `mkdir -p research && cp <repo>/curriculum/03-research-agents/research/R-0001.md research/` (`athf init` does not create `research/`). ATHF finds research by filename (`R-*.md` under `research/`), so there is nothing to index: `athf research list` shows R-0001, `--research R-0001` links to it. Copy `staff/DEMO_CLAUDE.md` in as `CLAUDE.md`. *No H-0007, and no other R- docs*, or ATHF hands out H-0008 / R-0002.
> 2. Check: `athf hunt validate` → 6 valid; `athf research list` → R-0001, Hunts –.
> 3. Splunk Web logged in, in a second window, time picker **All time** or **Last 30 days**, so you can flip terminal ↔ Splunk. No LLM key for `athf research` needed: research is pre-seeded, and Claude never touches Splunk here.
> 4. Launch Claude with `staff/demo-claude.sh` from inside the workspace: vanilla, only the ATHF MCP server, `athf hunt new` pre-approved. **Not** your own Claude Code in an attendee workspace: a dry run there, with personal plugins and no hunt opened first, answered "update the hunt" by editing H-0005, and H-0007 never appeared.
> 5. Prompts are below and in the staff cheat sheet; paste them, don't retype them. **How to paste results back:** in Splunk's results table, drag-select from the header row to the last row, copy, paste under `Result from Splunk:`. Tabs or spaces, either works. Times show in your Splunk timezone; that doesn't matter. Each search's exact rows are below, if copying from the table fights you.
>
> Artifacts: TIER_MANIFEST A5 (git-hook persistence) and A8 (lookalike on the shared hosting block). Every value below was checked on the Nov corpus with `staff/splunk-search.sh`. The whole flow (create H-0007, seven searches pasted back one at a time, update H-0007) was run twice through headless vanilla Claude Code (Sonnet 5, Bedrock, `demo-claude.sh -p`, then `--continue` per step) on 2026-11-11, and the SPL it wrote was run as-is and its rows pasted back. Both runs ended with a valid H-0007 linked to R-0001.

## Opening — research is the step before the hunt (3 min)

**Say:** Modules 1 and 2 were hunting: you had a hypothesis and tested it. Research is the step before that. It reads what's known about a behavior and tells you what to look for. In ATHF they're two objects: a research doc ( `R-0001` ) proposes, a hunt ( `H-0007` ) tests, and the hunt links back to the research that started it.

**Say:** Today Claude writes the searches and I run them. That's L3: the agent drafts, a human reviews and executes. In Module 4 Claude runs them itself.

## Beat 1 — Research the open thread (5 min)

**Say:** H-0005 is still open. We know WS-BUILD-01 beacons every ~5 minutes. We don't know what keeps it running, or what else the actor runs. The events are in Splunk; what we're missing is *what to look for*.

**Show** (on screen; don't run it, it was run before class):

```
athf research new \
  --topic "Periodic outbound to a CDN-lookalike domain after a build-CLI "\
"install on a developer workstation: how the node beacon persists "\
"through developer tools like git, npm or the IDE, and how to find "\
"related attacker infrastructure" \
  --technique T1071.001 \
  --depth basic
```

**Say:** This takes about 30 seconds and four LLM calls, so I ran it before class. Five steps: how does this normally work, how do attackers abuse it, which log fields show it, have we hunted it before, then a summary. The topic reuses H-0005's words on purpose: step four searches your hunts, and it only finds H-0005 if the words match.

**Following along?** Read `research/R-0001.md` in your repo. To run research yourself later, `athf research new` needs an Anthropic or OpenAI API key, AWS Bedrock credentials, or a local Ollama. A Claude subscription alone won't work.

**Show:** open `research/R-0001.md` in the editor. Read four lines:

1. **Related Work:** `H-0005` . It read *your* hunt.
2. **System Research:** *"...core.hooksPath pointing to a shared directory..."* → persistence.
3. **Tradecraft:** *"...unexpected outbound Node.js process network connections from IDE or git-related parent processes"* → the search.
4. **Tradecraft:** *"...same ASN/registrar ... share JA3/JA3S fingerprints"* → infrastructure reuse. If we don't have JA3: resolve the server name to its IP and look next door.

**Guiding question (interpretation):** *Which claims can you check in our data, and which are guesses? Look at "Data Source Availability" and the beacon interval it expects.*

> **Answer key — staff + self-study**
>
> Model answer (interpretation Q): checkable leads are git hooks / `core.hooksPath`, node started by git, and a second lookalike on the same hosting. Guesses: "Network Connections: Limited/Unavailable" (wrong: we have eCAR FLOW, the proxy and Zeek), "every 30–120 minutes" (ours is ~5 min), and JA3 (Beat 4 checks). Every line is a lead until Splunk says so. If asked "is this doc real?": it is the unedited output of that exact command (ATHF 0.24.0, Bedrock Sonnet, basic depth, no web search). 2 of 2 rehearsal runs named git hooks and shared hosting; the committed doc is one of them.

**Why the topic names git:** a dev box runs git, npm and an IDE all day. Those are the tools a backdoor would hide in.

## Beat 2 — Open H-0007 from the research (5 min)

**Say:** We create the hunt first, before any search. The research gives us the hypothesis and what to look for; the hunt is where the searches and results will land. And it gives "update the hunt" one obvious meaning later: H-0007, not H-0005.

**Type** (to Claude):

```
Read research/R-0001.md and hunt H-0005 (the open beacon thread). Before we
search anything, create the hunt this research starts, H-0007. Use the athf
hunt new CLI in the terminal (not the MCP tool, so the research link is
written both ways), with --non-interactive and --research R-0001. Title:
"Git-hook persistence relaunches the beacon; lookalike C2 domain on the same
hosting". Technique T1546; tactics persistence and command-and-control;
platform windows; data sources edr, proxy and network. Fill --hypothesis,
--threat-context, --actor, --behavior, --location and --evidence from the
research and H-0005, in plain text (no backticks or $ inside the quotes).
Leave --hunter off (the config has it). Then, in the new file, fill OBSERVE
(what normal looks like, what suspicious looks like, expected observables)
and set status to in-progress. Leave CHECK and KEEP as the template. Run athf
hunt validate. Only create the hunt — don't write any searches yet.
```

**Show:**

1. The `athf hunt new` command Claude builds: `--non-interactive --research R-0001`.
2. Its output: `Created H-0007` and `Linked H-0007 to research R-0001`.
3. The new file: `spawned_from: R-0001`, the hypothesis and ABLE table in LEARN, OBSERVE filled, CHECK and KEEP still the template.
4. `research/R-0001.md` now lists `linked_hunts: ['H-0007']`.

> **Answer key — staff + self-study**
>
> Rehearsal (2 runs): both read R-0001 and H-0005 (ATHF MCP tools), ran `athf hunt new --help`, then one multi-line `athf hunt new --non-interactive --research R-0001 …` with every flag, read the file, edited status and OBSERVE, ran `athf hunt validate H-0007` → valid. 11 turns, ~1 min, no permission prompts. Its LEARN/OBSERVE text is longer than the staff reference; that's fine. **Why each clause is there** (each one closes a way this step fails quietly):
>
> - *"Before we search anything"*: in an earlier dry run nothing was created until the end, so "update the hunt with these results" edited H-0005 and H-0007 never existed.

- *"the CLI, not the MCP tool"*: ATHF's MCP `athf_hunt_new` creates the file with `spawned_from: R-0001` but does **not** write `linked_hunts` into R-0001, and leaves hunter unfilled.
- *"--non-interactive"*: without it `athf hunt new` prompts for input, gets none from Claude's shell, and exits `Aborted!`.
- *"no backticks or $"*: inside double quotes the shell would run them, and the command needs approval or breaks. **If it goes sideways:** run the command below yourself, then paste the **H-0007 OBSERVE** block. If Claude created H-0007 but LEARN is off, paste **H-0007 LEARN**.

*Lines too long for the page wrap here. To copy this block, use the Markdown handout.*

```
athf hunt new --non-interactive --research R-0001 \
  --title "Git-hook persistence relaunches the beacon; lookalike C2 domain on the same hosting" \
  --technique T1546 \
  --tactic persistence --tactic command-and-control \
  --platform windows \
  --data-source edr --data-source proxy --data-source network \
  --actor "CRON GOBLIN" \
  --hypothesis "CRON GOBLIN keeps the H-0005 beacon (node loader.js --beacon on WS-BUILD-01) running through a
developer-tool hook such as a global git core.hooksPath, and runs more than one CDN-lookalike domain on the same
hosting; evidence is git launching a non-git child in eCAR and a second domain on the C2's IP block in Zeek ssl." \
  --threat-context "R-0001: dev-tool persistence (git hooks, core.hooksPath) and infrastructure reuse (same
hosting/ASN, JA3) for CDN-lookalike C2" \
  --behavior "a developer-tool hook (git core.hooksPath, an npm script or the IDE) that restarts the node beacon; a
second lookalike domain on the C2's hosting" \
  --location "WS-BUILD-01 (10.60.10.11), egress via PROXY-01" \
  --evidence "eCAR process lineage (ppid -> pid) from git to node; Zeek ssl server_name and id.resp_h around the
C2's IP; proxy CONNECTs by client"
```

Then `status:` and **Status:** `planning` → `in-progress`.

**H-0007 LEARN.** Select from `### Hypothesis Statement` down to the end of the ABLE table, and paste this over it.

*Lines too long for the page wrap here. To copy this block, use the Markdown handout.*

```
### Hypothesis Statement

CRON GOBLIN keeps the H-0005 beacon (node loader.js --beacon on WS-BUILD-01) running through a developer-tool hook
such as a global git core.hooksPath, and runs more than one CDN-lookalike domain on the same hosting; evidence is
git launching a non-git child in eCAR and a second domain on the C2's IP block in Zeek ssl.

### Threat Context

R-0001: dev-tool persistence (git hooks, core.hooksPath) and infrastructure reuse (same hosting/ASN, JA3) for CDN-
lookalike C2

### ABLE Scoping

| **Field**   | **Your Input** |
|-------------|----------------|
| **Actor** *(Optional)* | CRON GOBLIN |
| **Behavior** | a developer-tool hook (git core.hooksPath, an npm script or the IDE) that restarts the node
beacon; a second lookalike domain on the C2's hosting |
| **Location** | WS-BUILD-01 (10.60.10.11), egress via PROXY-01 |
| **Evidence** | eCAR process lineage (ppid -> pid) from git to node; Zeek ssl server_name and id.resp_h around the
C2's IP; proxy CONNECTs by client |
```

**H-0007 OBSERVE.** Select from `### What Normal Looks Like` down to the `- **Authentication:**` line, and paste this over it.

*Lines too long for the page wrap here. To copy this block, use the Markdown handout.*

```
### What Normal Looks Like

git runs `status`, `pull`, `log` and the occasional `commit`, and launches nothing but git. Proxy CONNECTs go to
well-known SaaS and update hosts.

### What Suspicious Looks Like

git launching a shell or node; `core.hooksPath` pointed at a folder outside any repo; a second young lookalike
domain on the same IP block as a known C2.

### Expected Observables

- **Processes:** git → cmd.exe or node; `node … loader.js --beacon` (H-0005's beacon, pid 5996)
- **Network:** node OUTBOUND flows to PROXY-01 (10.60.20.30:3128); Zeek ssl sessions to cdn-packmule-telemetry.net
and its neighbours on the same IP block
- **Files:** the folder that holds `loader.js`, and any hook scripts beside it
- **Registry:** N/A
- **Authentication:** proxy user `SUDOSHIPPING\alice.chmod` (H-0005)
```

`athf hunt validate` → 7 valid.

## Beat 3 — One search at a time: what starts the beacon? (9 min)

**Say:** Now the loop. I ask Claude for **one** search. I read it, paste it into Splunk, run it, and paste the result rows back. Claude says what they mean, then writes the next one. One search per turn keeps every step small enough for me, and you, to check.

**The loop** (every turn after the first):

```
Result from Splunk:

<paste the result rows, header included>

What does this tell us, in two or three lines? Then write the next single
search: <the next search, from this handout>. Same rules: don't run it,
one code block, one line of why. Only this one search — don't
investigate further.
```

### S1 — What did git launch, on every host?

**Type** (to Claude):

```
Now the first search, for the research's first lead: git hooks, and node
started from git-related parent processes. You can't reach Splunk in this
module: write the search, don't run it. I'll paste it into Splunk and paste
the result back to you. Write ONE search: what git launched, on every host
(eCAR process creates whose parent image is git.exe), as a table with _time,
hostname, principal, pid, ppid, image and command line. Reuse the index,
sourcetype and field names from our hunts' CHECK sections. No subsearches, no
comments in the SPL, one code block, one line of why. Only write this one
search — don't investigate further.
```

**Claude writes** (review it, then paste into Splunk):

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
properties.parent_image_path="*\\git.exe"
| table _time hostname principal pid ppid properties.image_path properties.command_line
```

**Answer key — staff + self-study**

Rehearsal (2 runs): both grepped the hunts for the command-line field name, then wrote exactly this search and stopped. **Review before you paste:** `index=thrunt`, the eCAR sourcetype, `"*\\git.exe"` with the escaped backslash. Expected: **one row in the whole fleet** (446 git process events on 9 hosts; git launches nothing else anywhere). Paste back:

*Lines too long for the page wrap here. To copy this block, use the Markdown handout.*

```
_time hostname principal pid ppid properties.image_path properties.command_line
2026-11-06 14:14:37.708 WS-BUILD-01 alice.chmod 5968 5964 C:\Windows\System32\cmd.exe cmd.exe /k
C:\Users\alice.chmod\AppData\packmule\hooks\pre-commit.cmd
```

### S2 — What else ran from a hooks folder on that host?

**Type** (to Claude): `Result from Splunk:`, the S1 rows, then:

```
What does this tell us, in two or three lines? Then write the next single
search: everything on that host whose command line mentions hooks, in time
order, with pid, ppid, parent image and command line. Same rules: don't run
it, one code block, one line of why. Only this one search — don't
investigate further.
```

**Claude writes:**

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
hostname="WS-BUILD-01" properties.command_line="*hooks*"
| sort _time
| table _time pid ppid properties.parent_image_path properties.command_line
```

**Answer key — staff + self-study**

Claude concluded (both runs): git ran a hook from a folder in alice's AppData, not the repo's own `.git\hooks`, so something redirected git's hook path. Search as above (one run put `sort` after `table`: same rows). Expected: three rows within 6 s, ~75 min after the npm install, chained by ppid = pid. Paste back:

*Lines too long for the page wrap here. To copy this block, use the Markdown handout.*

```
_time pid ppid properties.parent_image_path properties.command_line
2026-11-06 14:14:32.146 5964 5612 C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe git config --global
core.hooksPath C:\Users\alice.chmod\AppData\packmule\hooks
2026-11-06 14:14:37.708 5968 5964 C:\Program Files\Git\cmd\git.exe cmd.exe /k
C:\Users\alice.chmod\AppData\packmule\hooks\pre-commit.cmd
2026-11-06 14:14:37.976 5996 5968 C:\Windows\System32\cmd.exe node
C:\Users\alice.chmod\AppData\packmule\hooks\loader.js --beacon
```

**Say (honest shortcut):** "Look at the hook's parent: it's the `git config` process. In the real world a pre-commit hook fires on every `git commit`; there are zero `git commit` events on WS-BUILD-01. Our data

> generator took a shortcut and fired the hook straight from the config change. The lesson holds: git launched a shell from a folder in AppData, and that shell launched node."

### S3 — Is that node process the beacon from H-0005?

**Type** (to Claude): `Result from Splunk:` , the S2 rows, then:

```
What does this tell us, in two or three lines? Then write the next single
search: node's outbound flows on that host (eCAR FLOW, OUTBOUND; filter on
the node image, not a pid), counted by pid and destination IP and port, with
the first time. Same rules: don't run it, one code block, one line of why.
Only this one search — don't investigate further.
```

**Claude writes:**

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=FLOW properties.direction=OUTBOUND
hostname="WS-BUILD-01" properties.image_path="*node*"
| stats count min(_time) as first by pid properties.dst_ip properties.dst_port
```

> **Answer key — staff + self-study**
>
> Claude concluded (both runs): the full chain, PowerShell → `git config core.hooksPath` → `pre-commit.cmd` → `node loader.js --beacon` , pid 5996, the beacon pid from H-0005. Rehearsal: run 1 (before "not a pid" was in the prompt) filtered `pid=5996` ; run 2 wrote `properties.image_path="*\\node.exe"` with `earliest(_time)` . Same row either way; the image filter is the better check, because it shows *all* node traffic is that one pid. Expected: one row, 121 flows (H-0005's count), first ~16 min after the loader starts. A FLOW's `pid` is the process that opened it. Paste back:
>
> ```
> pid properties.dst_ip properties.dst_port count first
> 5996 10.60.20.30 3128 121 1793975427.394
> ```

**What it means.** Compare with Module 1's front door.

**Guiding question (decision):** *M1 found node launching a shell after an install. What's the same idea here, and what changed?*

> **Answer key — staff + self-study**
>
> Model answer (decision Q): the same shape as M1 ("a trusted dev tool launches something it never should"), with git instead of npm. And it explains H-0005: the hook started the beacon. Logs only, so `suspected` ; reading `~/.gitconfig` and the hooks folder on the host would confirm it.

## Beat 4 — Infrastructure reuse: does the actor run more than one domain? (9 min)

**Say:** Research's second lead: lookalike domains share hosting and TLS fingerprints (JA3). Before we pivot on a field, check we have it. Claude can't look, so it writes the check.

### S4 — Do we have JA3?

**Type** (to Claude): `Result from Splunk:` , the S3 row, then:

```
What does this tell us, in two or three lines? That closes the first lead.
The research's second lead: lookalike domains share hosting and JA3/JA3S TLS
fingerprints. Write the next single search: list the fields our Zeek SSL
data (sourcetype bro:ssl:json) actually has, so we can see whether ja3
exists. Same rules: don't run it, one code block, one line of why. Only this
one search — don't investigate further.
```

**Claude writes:**

```
index=thrunt sourcetype="bro:ssl:json" | fieldsummary | table field
```

> **Answer key — staff + self-study**
>
> Claude concluded (both runs): pid 5996's 121 flows match H-0005 exactly; the hook-launched node *is* the beacon. Search: run 2 wrote exactly this; run 1 added `count distinct_count | head 100` (same fields). Expected: no `ja3` / `ja3s` . Say: "research assumed a field we don't have; we keep the idea and use the server IP." Paste back (34 rows):
>
> ```
> field
> cert_chain_fuids{}
> cipher
> date_hour
> date_mday
> date_minute
> date_month
> date_second
> date_wday
> date_year
> date_zone
> established
> eventtype
> host
> id.orig_h
> id.orig_p
> id.resp_h
> id.resp_p
> index
> linecount
> punct
> resumed
> server_name
> source
> sourcetype
> splunk_server
> splunk_server_group
> ssl_history
> tag
> tag::eventtype
> timeendpos
> timestartpos
> ts
> uid
> version
> ```

### S5 — Where is the C2 hosted?

**Type** (to Claude): `Result from Splunk:` , the S4 field list, then:

```
What does this tell us, in two or three lines? Then write the next single
search: the server IP that cdn-packmule-telemetry.net resolves to in Zeek
SSL, with a count. Same rules: don't run it, one code block, one line of
why. Only this one search — don't investigate further.
```

**Claude writes:**

```
index=thrunt sourcetype="bro:ssl:json" server_name="cdn-packmule-telemetry.net"
| stats count by id.resp_h
```

> **Answer key — staff + self-study**
>
> Claude concluded (both runs): no `ja3` / `ja3s` ; pivot on `server_name` , `id.resp_h` (and `cert_chain_fuids` ) instead. Search as above (run 2 added `server_name` to the `by` ). Paste back:
>
> ```
> id.resp_h count
> 203.0.113.44 159
> ```
>
> 159, not 121: Zeek sees every client's tunnel through the proxy. Run 1 explained the gap as "TLS session reuse/retries": confidently wrong. Correct it out loud, and don't chase it; who else uses these domains is Module 4.

### S6 — What else is next door?

**Type** (to Claude): `Result from Splunk:` , the S5 row, then:

```
What does this tell us, in two or three lines? Then write the next single
search: every server_name in Zeek SSL on the addresses right next to that
IP (a /30, using cidrmatch; inside where, single-quote the dotted field as
'id.resp_h', but leave it unquoted in stats), with counts and IPs. Same
rules: don't run it, one code block, one line of why. Only this one search —
don't investigate further.
```

**Claude writes** (with the IP from S5):

```
index=thrunt sourcetype="bro:ssl:json"
| where cidrmatch("<C2 IP>/30", 'id.resp_h')
| stats count by server_name id.resp_h
```

> **Answer key — staff + self-study**
>
> Both runs wrote `cidrmatch("203.0.113.44/30", 'id.resp_h')` with `stats count by id.resp_h server_name`. Paste back:
>
> ```
> id.resp_h server_name count
> 203.0.113.44 cdn-packmule-telemetry.net 159
> 203.0.113.45 assets-packmule-cdn.com 5
> ```
>
> **Why the quoting clause is in the prompt:** in `where`/`eval`, a dot is the concatenation operator, so `cidrmatch("…/30", id.resp_h)` silently returns **0 rows**. An earlier rehearsal wrote exactly that; with only "single-quote it", the next run also quoted it in `stats count by server_name 'id.resp_h'`, also 0 rows. If Claude gets it wrong live, that's the review step doing its job: "zero rows, let's read it", paste the zero back, let it fix the quotes, re-run. If it uses a /24, three more appear: `static.docs-cdn.io` (.60, a benign CDN) and two S3 buckets (.70, .80). Say "wider block, more noise", ask for the /30.

### S7 — Who talks to it?

**Say:** Zeek only sees the proxy, so ask the proxy log.

**Type** (to Claude): `Result from Splunk:`, the S6 rows, then:

```
What does this tell us, in two or three lines? Then write the next single
search: in the proxy log, which internal client and user connected to the
new domain, with counts and first and last time. Same rules: don't run it,
one code block, one line of why. Only this one search — don't investigate
further.
```

**Claude writes** (with the new domain from S6):

```
index=thrunt sourcetype="apache:access:json" server="<the new domain>"
| stats count min(_time) as first max(_time) as last by client user
```

> **Answer key — staff + self-study**
>
> Claude concluded (both runs): a second lookalike, `assets-packmule-cdn.com`, one address over on the same /30. Search: `server="assets-packmule-cdn.com"` with `earliest`/`latest` (run 1 added `http_method=CONNECT`: same rows). Paste back:
>
> ```
> client user count first last
> 10.60.10.11 - 5 1794111464.179775 1794505416.640648
> 10.60.10.11 SUDOSHIPPING\alice.chmod 1 1793995211.674295 1793995211.674295
> ```
>
> **6** CONNECTs, all from `10.60.10.11` (WS-BUILD-01), about one a day for ~6 days, the first ~5.5h after the beacon starts; one of the five is a 403, hence Zeek's 5. Say: "only one of the six carries a proxy user; the client IP is what ties them to the host." In H-0005's C2 search it ranks **#77 of 150**: 6 against the beacon's 121. Nobody reads that far down.

**What it means.** Two lookalikes, next-door IPs, one client.

**Guiding question (interpretation):** *Why is "same hosting block" stronger evidence of the same actor than a similar name? When would it be weak?*

> **Answer key — staff + self-study**
>
> Model answer (interpretation Q): a domain name costs a few dollars and gets rotated; hosting is set up once and reused. It's weak when the block is shared hosting or a real CDN, where thousands of unrelated sites share IPs. That's why the block stays tiny and the client check matters. Research gave the idea (JA3); the data decided which field carries it.

## Beat 5 — Claude updates H-0007 (3 min)

**Say:** In Module 1 you wrote the hunt by hand. Here the assistant fills the hunt it opened, from the searches it wrote and the rows I pasted back.

**Type** (to Claude): `Result from Splunk:` , the S7 rows, then:

```
What does this tell us, in two or three lines? Then update H-0007 with what
we did: in CHECK, the seven searches I ran (one code block each under Hunting
Queries) and one Query Notes bullet per search with what it returned; in
KEEP, an Executive Summary and one suspected Findings row per finding (the
git-hook chain on WS-BUILD-01, and the second domain), plus Follow-up
Actions. Set status to completed and run athf hunt validate. Only edit
H-0007 — no new searches.
```

**Show:**

1. CHECK: the seven searches, each with what it returned.
2. KEEP: two `suspected` Findings rows.
3. `athf hunt validate` passes.

> **Answer key — staff + self-study**
>
> Rehearsal (2 runs): both said WS-BUILD-01 reached the second domain too (once as alice.chmod), then edited only H-0007: Data Source Information, seven `#### Query N` blocks each with its own notes, Query Performance, Executive Summary, two `suspected` rows (`WS-BUILD-01` hook chain; `assets-packmule-cdn.com · 203.0.113.45`), Follow-up Actions, status `completed`; validate passed, then `7 valid, 0 invalid`. 8–10 turns, ~1.5 min. R-0001 kept `linked_hunts: ['H-0007']`. **If it goes sideways**, paste these into H-0007 (they match `HUNTS_EXPECTED/H-0007.md`), then set `status:` and **Status:** to `completed`. Last resort: copy `HUNTS_EXPECTED/H-0007.md` over the file (fix `hunter:`), and add `H-0007` to R-0001's `linked_hunts` if Beat 2 didn't. `athf hunt validate` → 7 valid, 0 invalid (ATHF 0.24.0).
>
> **H-0007 CHECK, data source.** Paste over the four `- **…:** [` placeholder lines under **Data Source Information**.
>
> ```
> - **Index/Data Source:** `index=thrunt` · `evidenceforge:ecar:json` (PROCESS, FLOW) · `bro:ssl:json` (Zeek) · `apache:access:json` (PROXY-01)
> - **Time Range:** Last 30 days
> - **Events Analyzed:** 446 git process events, 121 beacon flows, 164 Zeek ssl sessions on the C2 block, 6 proxy CONNECTs
> - **Data Quality:** Good. No `ja3` field in Zeek ssl; Zeek sees the proxy as the client, so the real client comes from the proxy log.
> ```

**H-0007 CHECK, queries.** Paste over `[Your initial query]` and its fence lines.

~~~spl
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
properties.parent_image_path="*\\git.exe"
| table _time hostname principal pid ppid properties.image_path properties.command_line

index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
hostname="WS-BUILD-01" properties.command_line="*hooks*"
| sort _time
| table _time pid ppid properties.parent_image_path properties.command_line

index=thrunt sourcetype="evidenceforge:ecar:json" object=FLOW properties.direction=OUTBOUND
hostname="WS-BUILD-01" properties.image_path="*node*"
| stats count min(_time) as first by pid properties.dst_ip properties.dst_port

index=thrunt sourcetype="bro:ssl:json" | fieldsummary | table field

index=thrunt sourcetype="bro:ssl:json" server_name="cdn-packmule-telemetry.net"
| stats count by id.resp_h

index=thrunt sourcetype="bro:ssl:json"
| where cidrmatch("203.0.113.44/30", 'id.resp_h')
| stats count by server_name id.resp_h

index=thrunt sourcetype="apache:access:json" server="assets-packmule-cdn.com"
| stats count min(_time) as first max(_time) as last by client user
~~~

**H-0007 CHECK, notes.** Paste over the `**Query Notes:**` line and its two bullets.

*Lines too long for the page wrap here. To copy this block, use the Markdown handout.*

```
**Query Notes:**
- S1: git → child, every host: one row in the whole fleet, WS-BUILD-01 `cmd.exe /k …\packmule\hooks\pre-commit.cmd`
(pid 5968).
- S2: hooks: three rows, seconds apart, ~75 min after the install, chained by ppid = pid: `git config --global
core.hooksPath …\packmule\hooks` (pid 5964, from alice's PowerShell), then `pre-commit.cmd` (pid 5968, ppid 5964),
then `node …\hooks\loader.js --beacon` (pid 5996, ppid 5968). The hook's parent is the `git config` process itself
and there are no `git commit` events on WS-BUILD-01: the data generator fires the hook from the config change,
where a real pre-commit hook fires on each commit.
- S3: beacon flows: all 121 node flows to 10.60.20.30:3128 carry pid 5996, the loader; first one ~16 min after it
starts. The hook is what started the H-0005 beacon.
- S4: Zeek ssl has no `ja3`/`ja3s` field, so the research's JA3 pivot can't run; pivot on the hosting IP instead.
- S5: cdn-packmule-telemetry.net → 203.0.113.44 (159 sessions; Zeek counts every client through the proxy).
- S6: 203.0.113.44/30 → two domains: cdn-packmule-telemetry.net (.44, 159) and assets-packmule-cdn.com (.45, 5).
- S7: assets-packmule-cdn.com: 6 CONNECTs (one 403), all from 10.60.10.11, roughly one a day over ~6 days; only 1
carries the proxy user `SUDOSHIPPING\alice.chmod`, the other 5 log user `-`. Ranks #77 of 150 in H-0005's proxy
list, which is why nobody saw it.
```

**H-0007 KEEP.** Select from the `[Concise summary…]` line down to the second example row of the **Findings** table (the `suspected` one), and paste this over it.

*Lines too long for the page wrap here. To copy this block, use the Markdown handout.*

```
**Suspected.** On WS-BUILD-01, a global git `core.hooksPath` points at `AppData\packmule\hooks`; its pre-commit
hook starts `node loader.js --beacon`, the process behind all 121 beacon flows in H-0005, and it will start again
on every commit. The C2's hosting block also serves a second lookalike, assets-packmule-cdn.com, reached 6 times
over ~6 days from the same host (one CONNECT carries alice.chmod's proxy login).

### Findings

Things you're reporting as malicious. Only `confirmed` and `suspected` belong here.

| **Verdict** | **Subject** | **Ticket** | **Evidence** | **Independent Confirmation** |
|-------------|-------------|------------|--------------|------------------------------|
| `suspected` | WS-BUILD-01 · alice.chmod | N/A | eCAR: git config core.hooksPath (pid 5964) → pre-commit.cmd
(5968) → node loader.js --beacon (5996), linked by ppid; pid 5996 owns all 121 beacon flows | None — telemetry only
|
| `suspected` | assets-packmule-cdn.com (203.0.113.45) | N/A | Zeek ssl: same /30 as the C2 (203.0.113.44); proxy:
6 CONNECTs from 10.60.10.11 over ~6 days (1 with proxy user alice.chmod) | None — telemetry only |
```

**H-0007 FOLLOW-UP.** Under **Follow-up Actions**, replace the two example lines with:

*Lines too long for the page wrap here. To copy this block, use the Markdown handout.*

```
- [ ] Who else CONNECTs to cdn-packmule-telemetry.net or assets-packmule-cdn.com? (proxy log, by server; Zeek and
eCAR only ever show PROXY-01 talking to the block)
- [ ] Review `~/.gitconfig` and `AppData\packmule\hooks` on WS-BUILD-01; reset `core.hooksPath`
- [ ] Close H-0005's open thread with a link to this hunt
```

**Guiding question (reflection):** *Why `suspected`, not `confirmed`, even though every search lined up?*

**Answer key — staff + self-study**

Model answer (reflection Q): ATHF's evidence gate. Logs alone are `suspected`; `confirmed` needs something from outside them, e.g. reading `~/.gitconfig` and the hooks folder on WS-BUILD-01. The agent doesn't get to skip the gate either.

## Recap (1 min)

**Guiding question (reflection):** *Module 1 took 30 minutes by hand: seven searches, three copied IDs, one process tree and one open beacon. What did the last half hour produce, and what did the agent actually contribute?*

**Answer key — staff + self-study**

Model answer (reflection Q): seven searches across three log sources, two findings (the persistence that explains H-0005, and a second C2 domain nobody had seen), and a validated hunt with the SPL in it, linked to its research. Research gave *what to look for*; Claude opened the hunt, wrote each search and read each result; a human read and ran every search; Splunk decided what was real, including dropping JA3. Bridge to Module 4: Claude runs the searches itself, starting from H-0007's follow-up: who else CONNECTs to cdn-packmule-telemetry.net or assets-packmule-cdn.com (proxy log, by server)?

## Stop here — on the presenter's screen

```
athf hunt list
```

```
athf research list
```

> **Answer key — staff + self-study**
>
> Expected: `hunt list` shows H-0004 … H-0007 plus the 3 examples; `research list` shows R-0001 with Hunts `H-0007`.

**Next:** Module 4. Claude runs the searches itself, and uses your hunt history to decide what to hunt next.